

Banco de Dados I

Ronierison Maciel

Senac

Fevereiro 2026



Nome: Roni

Formação: Mestre em Ciência da Computação

Ocupação: Pesquisador, Professor e Escovador de bits

Hobbies: Jogar cartas, ficar com a família

Interesses: Carros, DevSec, DS e ML

Email: ronierison.maciел@pe.senac.br

GitHub: <https://github.com/0x03c1>

- 1 Módulo 1 – Introdução a Bancos de Dados e MySQL
- 2 Módulo 2 – Modelo Relacional e SQL Básico
- 3 Módulo 3 – SQL Intermediário: Filtros, Ordenação e Agregação
- 4 Módulo 4 – Modelagem de Dados e Diagramas ER
- 5 Módulo 5 – Normalização de Dados
- 6 Módulo 6 – Estrutura de Armazenamento
- 7 Módulo 7 – Indexação e Otimização de Consultas
- 8 Módulo 8 – Transações e Propriedades ACID
- 9 Módulo 9 – Triggers e Automação
- 10 Módulo 10 – Controle de Concorrência

Tópicos:

- Visão geral sobre BD e SGBD, instalação do MySQL
- Modelos de Dados, esquemas e arquiteturas
- Linguagens e interfaces de SGBDs, criação de tabelas

Objetivo:

- Introduzir os conceitos fundamentais de Banco de Dados e realizar a configuração inicial do MySQL

O que são Bancos de Dados (BD)?

Um Banco de Dados é uma coleção organizada de dados, tipicamente armazenados e acessíveis eletronicamente.

- **Exemplo:** Catálogo de produtos de uma loja, lista de alunos de uma escola.

CLIENTE	
CODIGO	NOME
1234	CARLOS
5678	JOÃO
9101	PEDRO
1213	MARIA

VENDEDOR	
CODIGO	NOME
11	CARMEM
12	DJANIRA
13	ZECA
14	MARIO

PRODUTO	
CODIGO	DESCRICAO
123	LAPIS
456	CANETA
789	PAPEL A4
101	TESOURA
123	BORRACHA
141	LIVRO

CONTÉM		
PEDIDO	PRODUTO	QUANTIDADE
100/05	123	10
100/05	789	20
101/05	456	30
102/05	456	40
103/05	101	50
103/05	121	60
103/05	141	70
104/05	456	80

PEDIDO			
NUMERO	DATA	VENDEDOR	CLIENTE
100/05	01/01/05	12	5678
101/05	01/02/05	11	9101
102/05	01/03/05	13	1213
103/05	01/04/05	14	1234
104/05	01/05/05	12	1213

Figure: Tabelas

O que é um Sistema de Gerenciamento de Banco de Dados (SGBD)?

Conteúdo:

- **Definição:** Um SGBD é um software que permite a criação, gestão, manipulação e controle de acesso a bancos de dados.
- **Funções:** Controle de concorrência, recuperação de falhas, segurança e integridade de dados.

Exemplos: MySQL, PostgreSQL, Oracle, SQL Server.

Por que Bancos de Dados são importantes?

- Centralização e organização dos dados.
- Suporte à tomada de decisão.
- Eficiência e escalabilidade em operações de negócio.

Casos de Uso: Comércio eletrônico, sistemas de gerenciamento de clientes (CRM), sistemas de controle financeiro.

Visão geral dos SGBDs mais usados

- **MySQL**: Open source, amplamente usado em aplicações web.
- **PostgreSQL**: Avançado, com suporte a tipos de dados complexos.
- **Oracle**: Focado em grandes empresas, oferece alta performance e segurança.
- **SQL Server**: Solução da Microsoft, integrada com outras ferramentas da empresa.

Gráfico Comparativo: Popularidade dos SGBDs (baseado em pesquisas recentes).

Você Sabia?

O mercado global de bancos de dados movimentou mais de **US\$ 100 bilhões em 2024**, com projeção de crescimento anual de 14% até 2030. O MySQL é utilizado por mais de **39% dos desenvolvedores** no mundo, segundo a pesquisa Stack Overflow 2024.

Dados do Mundo Real

- O **Facebook** gerencia mais de **600 terabytes** de dados novos por dia em seus bancos MySQL.
- O **Banco do Brasil** processa mais de **70 milhões de transações diárias** usando SGBDs relacionais.
- Empresas que adotam SGBDs modernos reduzem em até **40%** o tempo de resposta em consultas.

Como instalar o MySQL

Passos:

- Baixar o MySQL Community Server do site oficial.
- Executar o instalador e seguir as instruções.
- Configurar a senha do root e as opções de segurança.
- Verificar a instalação via terminal.

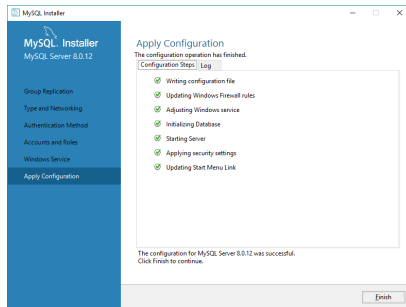


Figure: Instalação do MySQL

O que é MySQL Workbench?

Ferramenta GUI para modelagem de dados, desenvolvimento SQL e administração de servidores.

- Conectar ao servidor MySQL.
- Criar um banco de dados.
- Explorar as funcionalidades básicas.

Introdução aos Modelos de Dados

- **Modelos hierárquicos:** Dados organizados em uma estrutura de árvore.
- **Modelos em rede:** Dados organizados em gráficos, permitindo múltiplas relações.
- **Modelos relacionais:** Dados organizados em tabelas, a base do MySQL.



Figure: Hierárquico

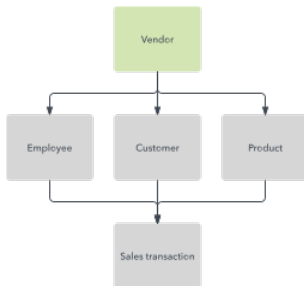


Figure: Rede

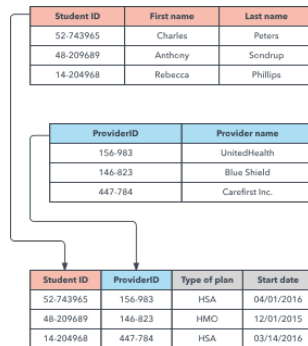


Figure: Relacional

Linha do Tempo

- **1960s:** Modelo Hierárquico – IBM IMS, usado pela NASA no programa Apollo.
- **1970:** Edgar F. Codd publica o artigo seminal sobre o **Modelo Relacional**.
- **1980s:** SGBDs relacionais dominam o mercado (Oracle, DB2).
- **2000s:** Surgem os bancos **NoSQL** (MongoDB, Cassandra) para Big Data.
- **2020s:** Bancos **NewSQL** e multimodelo ganham força (CockroachDB, Fauna).

Curiosidade

O artigo de Codd, “*A Relational Model of Data for Large Shared Data Banks*” (1970), é considerado um dos documentos mais influentes da história da computação e deu origem ao SQL.

Estrutura dos Bancos de Dados

- **Esquema:** A estrutura lógica de um banco de dados, definindo como os dados são organizados e inter-relacionados.

Arquiteturas:

- **Monolítica:** Todos os dados e serviços estão centralizados em um único sistema.
- **Cliente-Servidor:** Dados armazenados em servidores, acessados por clientes.
- **Distribuída:** Dados espalhados por múltiplos sistemas interconectados.

Linguagens e interfaces de SGBDs

Linguagens:

- **DDL** (Data Definition Language): Criação e modificação de estruturas de dados.
- **DML** (Data Manipulation Language): Inserção, atualização e exclusão de dados.

Interfaces:

- **CLI** (Command-Line Interface): Interação via comandos de texto.
- **GUI** (Graphical User Interface): Interação via interface gráfica, como o MySQL Workbench.

```
1 ALTER TABLE clientes ADD telefone VARCHAR(15);  
2 /* Comandos DDL */
```

```
1 SELECT nome, email FROM clientes WHERE data_cadastro > '2023-01-01';  
2 /* Comandos DML */
```

Definindo tabelas e tipos de dados

Tipos de Dados:

- Numéricos (INT, FLOAT).
- Texto (VARCHAR, TEXT).
- Data/Hora (DATE, TIMESTAMP).

Exemplo de criação de tabela:

```
1 CREATE TABLE alunos (  
2     id INT AUTO_INCREMENT PRIMARY KEY,  
3     nome VARCHAR(100),  
4     data_nascimento DATE  
5 );
```


Descrição:

- 1 Instalar o MySQL e MySQL Workbench em seus computadores.
- 2 Criar um banco de dados simples e tabelas com diferentes tipos de dados.
- 3 Inserir alguns registros e praticar comandos básicos de consulta.

Entrega: Enviar um relatório com capturas de tela e código SQL até a próxima aula.

O que vamos fazer na próxima semana?

- Revisão dos conceitos básicos de SQL.
- Introdução ao modelo de dados relacional.
- Primeiras operações com SQL (SELECT, INSERT, UPDATE, DELETE).

Tópicos:

- Conceitos do modelo de dados relacional e tabelas relacionais
- Operações básicas em SQL (SELECT, INSERT, UPDATE, DELETE)
- Transações e controle de transações em MySQL

Objetivo:

- Entender o modelo de dados relacional e realizar operações básicas utilizando SQL no MySQL.

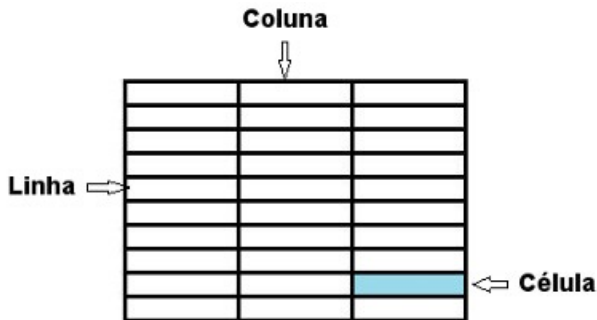
O que é o modelo de dados relacional?

Conceito:

- Modelo de dados que organiza informações em tabelas (também conhecidas como relações).

Elementos-chave:

- Tabelas:** Conjuntos de dados organizados em linhas e colunas.
- Linhas (Tuplas):** Cada linha representa um registro único.
- Colunas (Atributos):** Cada coluna representa um campo de dado dentro de um registro.



Estrutura de uma tabela relacional

- **Chave primária:** Coluna ou conjunto de colunas que identifica de forma única cada registro em uma tabela.
- **Chave estrangeira:** Coluna que cria uma ligação entre duas tabelas diferentes, referenciando a chave primária de outra tabela.
- **Índices:** Estruturas que melhoram a velocidade de operações de consulta nas tabelas.

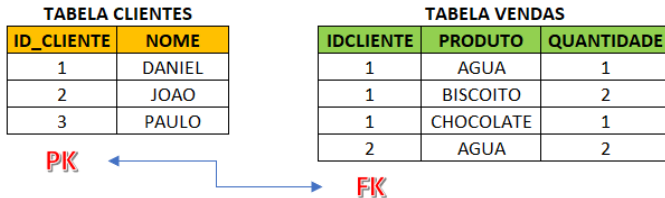


Figure: Foreign Key (FK) and Primary Key (PK)

Criando tabelas e definindo relações

Exemplo de SQL:

```
1 CREATE TABLE departamentos (  
2     id INT AUTO_INCREMENT PRIMARY KEY,  
3     nome VARCHAR(100)  
4 );  
5  
6 CREATE TABLE empregados (  
7     id INT AUTO_INCREMENT PRIMARY KEY,  
8     nome VARCHAR(100),  
9     departamento_id INT,  
10    FOREIGN KEY (departamento_id) REFERENCES departamentos(id)  
11 );
```

Prática: Criar tabelas e estabelecer relações no MySQL Workbench.

Introdução às operações básicas em SQL

- INSERT: Inserção de novos registros.
- SELECT: Consulta de dados.
- UPDATE: Atualização de registros existentes.
- DELETE: Exclusão de registros.

Inserindo dados em tabelas

```
1 INSERT INTO <tabela> (<coluna1>, <coluna2>, <coluna3>)  
2 VALUES (<valor1>, <valor2>, <valor3>);
```

Inserção de múltiplos registros

```
1 INSERT INTO <tabela> (<coluna1>, <coluna2>)  
2 VALUES  
3     (<valor1>, <valor2>),  
4     (<valor3>, <valor4>),  
5     (<valor5>, <valor6>);
```


Estrutura básica de uma consulta SQL

```
1 SELECT <coluna1>, <coluna2>  
2 FROM <tabela>  
3 WHERE <condicao>;
```

Principais cláusulas:

- WHERE – filtra linhas de acordo com uma condição.
- GROUP BY – agrupa registros com valores iguais.
- HAVING – filtra resultados após o agrupamento.
- ORDER BY – ordena os resultados da consulta.

Atualizando registros em uma tabela

```
1 UPDATE <tabela>  
2 SET <coluna1> = <valor1>, <coluna2> = <valor2>  
3 WHERE <condicao>;
```

Observações:

- SET define os novos valores das colunas.
- WHERE determina quais registros serão atualizados.
- Sem WHERE, **todas as linhas da tabela serão modificadas.**

Removendo registros de uma tabela

```
1 DELETE FROM <tabela>  
2 WHERE <condicao>;
```

Observações:

- DELETE remove linhas da tabela.
- WHERE define quais registros serão excluídos.
- Sem WHERE, **todas as linhas da tabela serão removidas.**

O que são transações em bancos de dados?

- Uma **transação** é um conjunto de operações (inserir, atualizar, excluir dados) que são executadas como uma única unidade de trabalho. Se todas as operações forem bem-sucedidas, as mudanças são confirmadas. Se alguma falhar, todas as mudanças são desfeitas (rollback), garantindo a integridade do banco de dados.

Propriedades ACID:

- **Atomicidade:** A transação é **tudo ou nada**; ou todas as operações são concluídas, ou nenhuma é.
- **Consistência:** A transação mantém o banco de dados em um estado consistente, respeitando todas as regras e restrições definidas.
- **Isolamento:** Transações são executadas de forma independente, sem interferir umas nas outras, como se fossem únicas no sistema.
- **Durabilidade:** Após a confirmação, as mudanças realizadas pela transação são permanentes, mesmo que ocorram falhas no sistema posteriormente.

Case: Nubank e o Poder do SQL

O Nubank processa mais de **1,5 bilhão de eventos por dia**. Toda operação de crédito, débito e transferência passa por transações SQL que garantem a **consistência financeira**. Um simples erro em um UPDATE sem WHERE custou a uma fintech brasileira mais de **R\$ 2 milhões** em 2022.

Dica Profissional

- **Nunca** execute UPDATE ou DELETE sem WHERE em produção.
- Sempre use START TRANSACTION + ROLLBACK ao testar alterações.
- O comando SELECT antes do UPDATE/DELETE é uma prática de segurança essencial.

Trabalhando com transações no MySQL

Comandos importantes:

- **START TRANSACTION:** Inicia uma nova transação.
- **COMMIT:** Confirma as mudanças realizadas pela transação.
- **ROLLBACK:** Reverte as mudanças realizadas pela transação.

Exemplo:

```
1 START TRANSACTION;  
2 UPDATE empregados SET salario = salario * 1.1 WHERE departamento_id = 1;  
3 COMMIT;
```

Essa query usa uma transação para aumentar em 10% os salários dos empregados do departamento `departamento_id = 1`. A transação começa com **START TRANSACTION**, a atualização é feita pelo **UPDATE**, e o comando **COMMIT** torna as mudanças permanentes.

Descrição:

- 1 Criar tabelas relacionadas no MySQL.
- 2 Inserir, atualizar e excluir registros utilizando SQL básico.
- 3 Realizar consultas complexas com filtros e agrupamentos.
- 4 Implementar transações envolvendo múltiplas operações.

Tópicos:

- Ampliar o conhecimento em SQL além dos comandos básicos já aprendidos: INSERT, SELECT, UPDATE e DELETE.
- Aprender a filtrar e ordenar dados utilizando as cláusulas WHERE e ORDER BY.
- Introduzir funções agregadas como COUNT, AVG, MIN e MAX para realizar cálculos em conjuntos de dados.
- Compreender o agrupamento de dados com a cláusula GROUP BY.

Filtrando dados com a cláusula WHERE

A cláusula WHERE permite selecionar apenas os registros que atendem a uma condição específica.

Exemplo 1 – Filtrando por valor específico

```
1 SELECT *  
2 FROM empregados  
3 WHERE departamento_id = 2;
```

Exemplo 2 – Operadores de comparação

```
1 SELECT *  
2 FROM empregados  
3 WHERE salario > 3000;
```

Exemplo 3 – Filtrando texto com LIKE

```
1 SELECT *  
2 FROM empregados  
3 WHERE nome LIKE 'A%';
```

Considere a tabela `empregados`, que possui o campo `data_admissao`.

Desafio:

Escreva uma consulta SQL que selecione os empregados admitidos após a data 2026-01-01.

Dica: Utilize a cláusula `WHERE` com um operador de comparação.

Ordenando resultados com ORDER BY

A cláusula ORDER BY organiza os resultados de uma consulta de acordo com uma ou mais colunas.
Ordem crescente (padrão)

```
1 SELECT *  
2 FROM empregados  
3 ORDER BY nome;
```

Ordem decrescente

```
1 SELECT *  
2 FROM empregados  
3 ORDER BY salario DESC;
```

Ordenação por múltiplas colunas

```
1 SELECT *  
2 FROM empregados  
3 ORDER BY departamento_id, nome;
```

Considere as tabelas departamentos e empregados.

Desafio:

- 1 Liste os departamentos ordenados pelo id em ordem decrescente.
- 2 Liste os empregados ordenados pela data_admissao, do mais recente para o mais antigo.

Funções agregadas realizam cálculos sobre um conjunto de registros e retornam um único valor.

Principais funções agregadas:

- **COUNT** – conta o número de registros.
- **AVG** – calcula a média de valores.
- **MIN** – retorna o menor valor.
- **MAX** – retorna o maior valor.
- **SUM** – calcula a soma de valores.

Exemplo:

```
1 SELECT COUNT(*)  
2 FROM empregados;
```

Agrupamento com GROUP BY

A cláusula GROUP BY agrupa registros que possuem o mesmo valor em uma ou mais colunas, permitindo aplicar funções agregadas a cada grupo.

Exemplo 1 – Número de empregados por departamento

```
1 SELECT departamento_id, COUNT(*) AS total_empregados
2 FROM empregados
3 GROUP BY departamento_id;
```

Exemplo 2 – Salário médio por departamento

```
1 SELECT departamento_id, AVG(salario) AS salario_medio
2 FROM empregados
3 GROUP BY departamento_id;
```

Exemplo 3 – Maior salário por departamento

```
1 SELECT departamento_id, MAX(salario) AS maior_salario
2 FROM empregados
3 GROUP BY departamento_id;
```

Considere a tabela `empregados`, que possui os campos `salario` e `departamento_id`.

- 1 Liste o número total de empregados na empresa.
- 2 Calcule a média salarial geral, bem como o menor e o maior salário entre todos os empregados.
- 3 Liste cada departamento juntamente com o total de salários pagos em cada um deles.

O que iremos aprender na próxima semana?

- Modelagem de Dados com ER.
- Introdução ao projeto de banco de dados relacional.
- Aplicação de UML na modelagem de banco de dados.

Por que dominar SQL é estratégico?

Você Sabia?

Segundo o LinkedIn, **SQL é a habilidade técnica mais demandada** em vagas de tecnologia e dados desde 2020. Profissionais que dominam SQL intermediário ganham em média **30% a mais** do que aqueles com conhecimento apenas básico (Glassdoor, 2024).

Impacto no Dia a Dia

- O **iFood** usa funções agregadas (COUNT, AVG) para gerar relatórios em tempo real sobre milhões de pedidos diários.
- Analistas de dados no **Magazine Luiza** utilizam GROUP BY com HAVING para identificar padrões de compra e otimizar estoques.
- Uma consulta SQL bem otimizada com WHERE indexado pode executar em **milissegundos** o que uma busca linear levaria **minutos**.

Tópicos abordados:

- Modelo Entidade–Relacionamento (ER)
- Projeto de Banco de Dados Relacional
- Modelagem utilizando UML

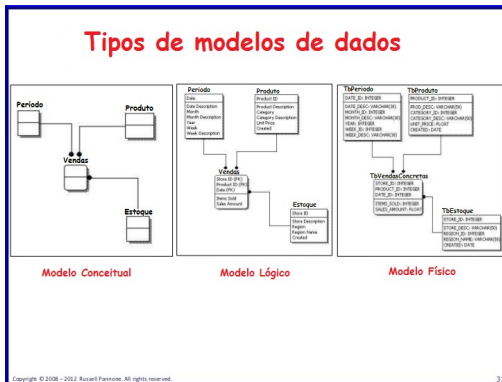
Objetivos de aprendizagem:

- Compreender os conceitos fundamentais do modelo ER.
- Representar entidades, atributos e relacionamentos.
- Aplicar técnicas de modelagem de dados utilizando ER e UML.

que é modelagem de dados?

Conceito de modelagem de dados

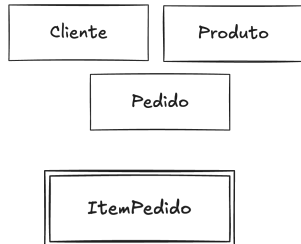
- **Definição:** A modelagem de dados é o processo de criar um modelo visual das informações que serão armazenadas em um banco de dados.
- **Objetivo:** Estruturar os dados de forma que possam ser armazenados, acessados e gerenciados de maneira eficiente.



Existem maneiras diferentes de apresentar um modelo Entidade Relacionamento.

- São notações que variam na utilização de símbolos e convenções.
 - Notação de Peter Chen;
 - Notação de James Martin;
 - Notação de Peter Chen adaptada por Carlos A. Heuser;

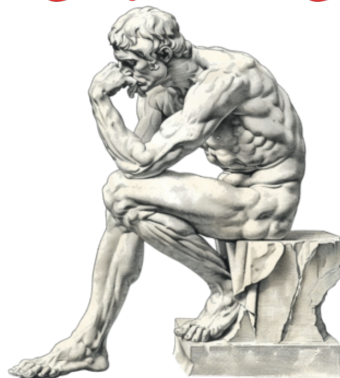
A notação de Peter Chen é muito difundida e utilizada e é caracterizada por sua simplicidade, representam objetos ou conceitos do mundo real que possuem existência própria no contexto do sistema.



Entidades: Representam objetos ou conceitos do mundo real que possuem existência própria no contexto do sistema.

Entidade fraca: Não possui um identificador único próprio e dependem de uma entidade forte para sua identificação. Item de Pedido (depende do Pedido), Dependente (depende do Funcionário).

O que é uma entidade?





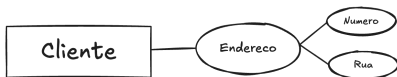
Simples: Representam as características ou propriedades das entidades.



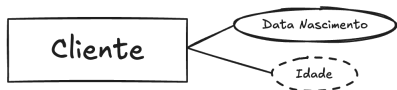
Chave primária: Identificam unicamente cada entidade dentro de um conjunto, funcionando como chave primária para distinguir instâncias individuais.



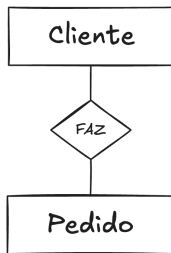
Multivalorados: Podem ter múltiplos valores para uma única entidade, como **telefones** em um cliente que possui vários números



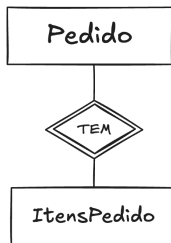
Composto: Podem ser decompostos em partes menores com significado próprio, como Endereço dividido em **rua, número, cidade** e **CEP**.



Derivado: Cujo valor é calculado a partir de outros atributos existentes, como **Idade** derivada da **Data de Nascimento**.



Relacionamento: Associação entre duas ou mais entidades que descreve como elas interagem ou se relacionam no modelo de dados, representada por um losango no diagrama ER.

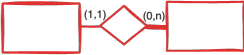


Relacionamento identificador: É um tipo de relacionamento onde uma entidade fraca depende de uma entidade forte para sua identificação; representado por um losango com bordas duplas, indica que a chave primária da entidade fraca inclui a chave primária da entidade forte.

- O professor Dr. Carlos Alberto Heuser, da UFRGS, adaptou a notação original de Peter Chen com o objetivo de simplificar os modelos Entidade-Relacionamento. Heuser introduziu uma notação que facilita a distinção entre atributos simples e derivados (aqueles calculados a partir de outros atributos)
 - A ferramenta brModelo foi criada pelo Heuser e utiliza essa notação.

Colocando a mão na modelagem!

- Crie um diagrama ER para um sistema de gerenciamento de uma locadora de filmes. Considere as seguintes entidades: Filme, Cliente, e Locação. Defina os atributos principais para cada entidade e as relações entre elas.

Conceito	Símbolo
Entidade	
Relacionamento	
Atributo	
Atributo identificador	
Relacionamento identificador (Entidade fraca)	
Generalização/especialização	
Entidade associativa	

Cardinalidade



Cardinalidade: Quantos elementos se relacionam? Um para um, um para muitos ou muitos para muitos.

- **Relação 1:1 (um para um):**

- Significa que um registro em uma tabela A está associado a no máximo um registro em uma tabela B, e vice-versa.



- **A notação:** O "1:1" significa que para cada instância de uma entidade (como uma pessoa), há uma e somente uma instância associada na outra entidade (passaporte).

- **Relação 1:N (um para muitos):**

- Um registro em uma tabela A pode estar relacionado a vários registros em uma tabela B, mas cada registro em B está relacionado a apenas um registro em A.



- **A notação:** O "1" (onde N pode ser qualquer número) significa que uma instância da entidade A pode estar relacionada a muitas (ou seja, múltiplas) instâncias da entidade B.

- **Relação N:N (muitos para muitos):**

- Vários registros em uma tabela A podem estar relacionados a vários registros em uma tabela B. Esse tipo de relação normalmente é implementado usando uma tabela intermediária (tabela de junção) que referencia ambas as tabelas.



- **A notação:** O "N" (onde N e N podem ser quaisquer números) indica que muitas instâncias de uma entidade A podem estar relacionadas a muitas instâncias de uma entidade B.

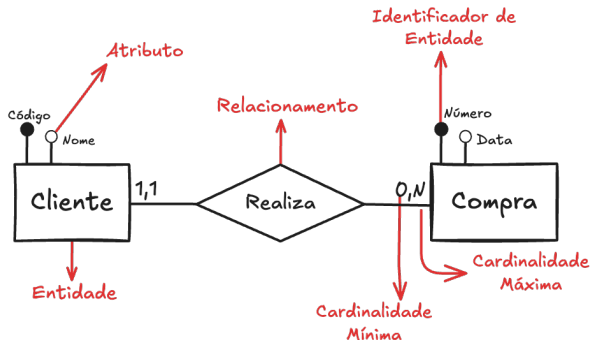
Entidade-Relacionamento (ER)

Conceito:

- O MER é uma representação gráfica com a estrutura lógica de um BD.

Componentes:

- **Entidades:** Objetos ou conceitos sobre os quais os dados são armazenados.
- **Atributos:** Características das entidades.
- **Relacionamentos:** Associações entre entidade.



Entidades:

- Representadas por retângulos. Exemplo: Clientes, Compra.

Atributos:

- Representados por elipses. Exemplo: Nome, Código.

Relacionamentos:

- Representados por losangos. Exemplo: Realiza, Contém, Faz.

Chave primária:

- Um atributo ou conjunto de atributos que identifica de forma única uma entidade.

Modelagem de um sistema de vendas

1 Entidades identificadas:

- Cliente: Atributos: ClienteID, Nome, Endereço.
- Produto: Atributos: ProdutoID, NomeProduto, Preço.
- Pedido: Atributos: PedidoID, DataPedido, ClienteID.

1 Relacionamentos:

- Cliente faz Pedido.
- Pedido contém Produto.

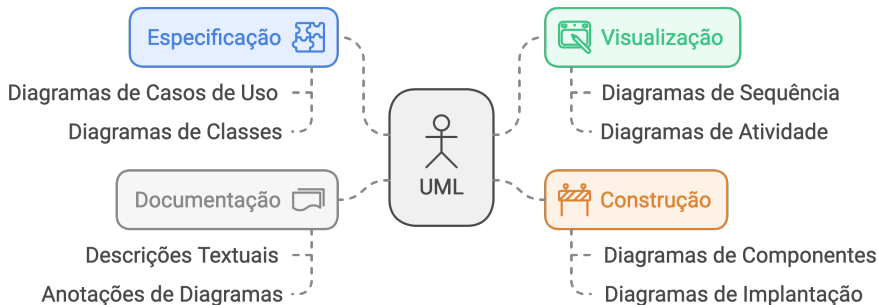
Você Sabia?

Segundo pesquisa da IBM, **70% dos problemas em sistemas** de banco de dados têm origem em uma **modelagem conceitual mal feita**. Corrigir erros de modelagem após a implementação custa até **100 vezes mais** do que corrigi-los na fase de projeto.

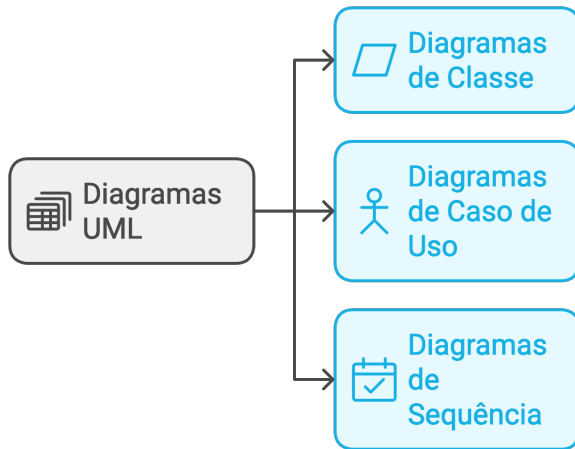
Casos de Sucesso com Modelagem ER

- O **SUS (Sistema Único de Saúde)** utiliza diagramas ER para modelar mais de **200 entidades** que gerenciam dados de 210 milhões de brasileiros.
- A **Amazon** redesenhou sua modelagem de dados em 2015, resultando em **40% de redução no tempo de consulta** de seu catálogo.
- A ferramenta **brModelo**, criada no Brasil pelo Prof. Heuser, é usada em mais de **500 instituições de ensino** em língua portuguesa.

UML é uma linguagem de modelagem padrão usada para especificar, visualizar, construir e documentar os componentes de sistemas de software.



Diagramas diversificados



Orientação a Objetos:

- Suporta princípios da orientação a objetos, facilitando a representação de classes, objetos, herança, polimorfismo, etc.

Notação Gráfica:

- Utiliza símbolos gráficos padronizados para representar elementos como classes, atributos, métodos, relacionamentos, etc.

Orientação a Objetos:

Princípios da Programação Orientada a Objetos

Polimorfismo

Capacidade de processar objetos de maneira diferente com base em seu tipo de dado



Herança

Mecanismo para criar novas classes a partir de classes existentes



Classes

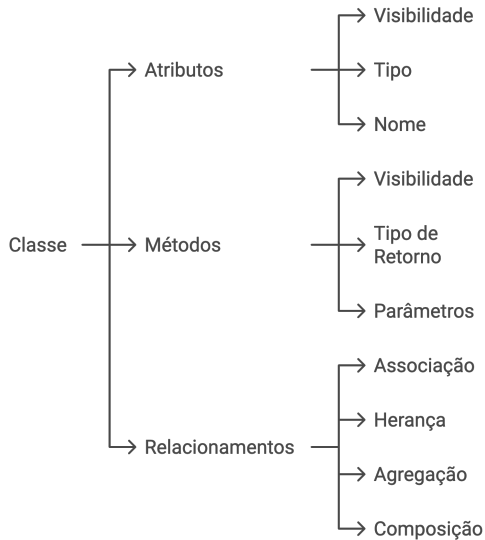
Blocos de construção fundamentais em POO definindo modelos de objetos



Objetos

Instâncias de classes representando entidades do mundo real





- **Classes:**

- Representadas por retângulos divididos em três partes: nome da classe, atributos e métodos.

- **Atributos:**

- Listados no segundo compartimento do retângulo da classe, geralmente com visibilidade (+, -, #), tipo e nome.

- **Métodos:**

- Listados no terceiro compartimento, também com visibilidade, retorno e parâmetros.

- **Relacionamentos:**

- Linhas conectando classes, podendo indicar associações, heranças, agregações, composições, etc., com multiplicidades indicadas próximo às extremidades das linhas.

Objetivo: Entender o que é o modelo lógico de banco de dados, sua importância, e como transformá-lo em uma representação formal que permita a implementação física no banco de dados.

- O que é um modelo lógico?
- É uma representação intermediária entre o modelo conceitual (ER - Entidade-Relacionamento) e o modelo físico. Ele traduz os elementos conceituais em um formato mais próximo de tabelas, chaves e colunas, mas ainda independente de um SGBD específico.
- Serve como uma ponte para garantir que as ideias do modelo conceitual sejam refletidas adequadamente na implementação final.

- **Entidades como tabelas:** Cada entidade do modelo conceitual se transforma em uma tabela no modelo lógico.
- **Atributos:** Os atributos tornam-se colunas nas tabelas.
- **Relacionamentos:** Definidos por meio de chaves estrangeiras (foreign keys), que ligam as tabelas entre si.
- **Normalização:** Processo que visa reduzir redundâncias e melhorar a consistência.

- **Modelo conceitual:** Abstrato, focado em representar entidades e relacionamentos de forma mais livre, muitas vezes usando diagramas ER.
- **Modelo lógico:** Mais próximo da implementação, com uma estrutura que já reflete tabelas, tipos de dados, e relações, mas sem especificar aspectos específicos de um SGBD.

Fluxo Profissional de Modelagem

- 1 **Modelo Conceitual** (ER): Comunicação com o cliente e stakeholders.
- 2 **Modelo Lógico**: Tradução para tabelas, chaves e restrições.
- 3 **Modelo Físico**: Implementação no SGBD com tipos de dados, índices e partições.

Dica do Mercado

Empresas como **TOTVS** e **Stefanini** exigem que toda modelagem passe pelas três fases antes da implementação. Pular etapas é a causa número 1 de retrabalho em projetos de banco de dados.

Exemplo conceitual: Um diagrama ER com as entidades "Cliente" e "Pedido", conectadas por um relacionamento "Faz".

- Exemplo lógico: Transformando as entidades em tabelas:
 - **Tabela Cliente:** ID_Cliente (PK), Nome, Endereço.
 - **Tabela Pedido:** ID_Pedido (PK), Data, ID_Cliente (FK).
 - **Relacionamento:** A FK em "Pedido" conecta cada pedido a um cliente.

Primeira Forma Normal (1NF):

- Elimina grupos repetidos, garantindo que cada coluna contenha valores atômicos (não divisíveis).
- Suponha uma tabela "Pedido" que contenha um campo "Itens" com múltiplos produtos em um único registro. Para normalizar, criar uma nova tabela chamada "Pedido_Item", onde cada produto do pedido seja representado em uma linha separada.

Segunda Forma Normal (2NF):

- Garante que todos os atributos não-chave dependam unicamente da chave primária, eliminando dependências parciais.
- Em uma tabela "Pedido" com atributos "ID_Pedido", "ID_Cliente", e "Endereço_Cliente", o atributo "Endereço_Cliente" não depende da chave primária "ID_Pedido", mas sim do "ID_Cliente". Para normalizar, mova o atributo "Endereço_Cliente" para a tabela "Cliente"

Terceira Forma Normal (3NF):

- Remove dependências transitivas, ou seja, nenhum atributo não-chave deve depender de outro atributo não-chave.
- Suponha uma tabela "Cliente" com os atributos "ID_Cliente", "Nome_Cliente", e "Cidade". Se houver também um campo "Região" que depende de "Cidade", a tabela não está em 3NF. Para normalizar, criar uma tabela separada para armazenar as informações de "Cidade" e "Região".

Você Sabia?

Segundo estudo da Oracle, bancos de dados **não normalizados** consomem até **60% mais espaço em disco** e apresentam **3x mais anomalias** de inserção, atualização e exclusão. A normalização é a base de todo banco relacional bem projetado.

Cenário Real

- O sistema do **Detran-PE** sofreu inconsistências graves por tabelas fora da 2NF, causando duplicação de registros de veículos em 2019.
- A **Netflix** mantém suas tabelas de catálogo em 3NF, mas usa **desnormalização controlada** em tabelas de cache para ganho de performance.
- **Desnormalizar sem critério** é um dos erros mais comuns de DBAs juniores — sempre normalize primeiro, depois otimize.

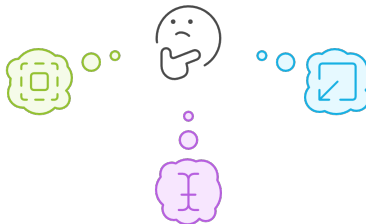
Como os dados são armazenados fisicamente?



Qual é a melhor forma de otimizar o desempenho do banco de dados MySQL?

Aumentar o tamanho da página

Permite armazenar mais registros em uma única página, reduzindo o número de leituras e gravações no disco.



Reduzir o tamanho do bloco

Permite acessar dados mais rapidamente, mas pode aumentar o número de leituras e gravações no disco.

Otimizar o tamanho do registro

Reduz o espaço ocupado por cada registro, permitindo armazenar mais registros em uma única página.

- Para explicar o conceito de páginas, observaremos quando os dados são inseridos em uma tabela, o MySQL agrupa essas linhas em páginas.

```
1 CREATE TABLE Alunos (  
2     id INT PRIMARY KEY AUTO_INCREMENT,  
3     nome VARCHAR(50),  
4     idade INT  
5 ) ENGINE=InnoDB;  
6  
7 INSERT INTO Alunos (nome, idade) VALUES ('João', 20), ('Maria', 22), ('Lucas', 19), (  
    'Ana', 21);
```

- O conceito de bloco é mais interno ao sistema operacional e ao mecanismo de armazenamento, mas pode ser compreendido como o agrupamento de páginas em uma estrutura lógica.

```
1 SHOW ENGINE INNODB STATUS;
```

- Um registro no MySQL contém todos os dados de uma linha e ocupa espaço dentro da página.

```
1 INSERT INTO Alunos (nome, idade) VALUES ('Carlos', 23), ('Júlia', 20), ('Fernando',  
2 25);  
3 SELECT * FROM Alunos WHERE idade > 20;
```

- Essa consulta busca registros específicos, e cada linha recuperada ocupa espaço dentro de uma página.
- Ao adicionar mais registros, a quantidade de páginas necessárias aumenta, o que influencia a organização do armazenamento físico.

Arquitetura InnoDB em Números

- Cada **página** no InnoDB tem tamanho fixo de **16 KB** por padrão.
- Um **extent** agrupa **64 páginas** consecutivas (1 MB).
- O **Buffer Pool** mantém páginas frequentemente acessadas em memória RAM, reduzindo I/O em disco em até **95%**.

Você Sabia?

O **Mercado Livre** otimizou sua configuração de páginas InnoDB e reduziu o tempo de resposta de consultas críticas em **50%** apenas ajustando o tamanho do Buffer Pool. Entender a estrutura física do armazenamento é essencial para DBA de alto nível.

- A indexação é uma técnica usada para acelerar o acesso aos dados, permitindo que o banco de dados encontre registros específicos sem precisar fazer uma leitura completa da tabela.
- Ao criar um índice, o banco de dados cria uma estrutura que mapeia rapidamente os registros aos seus locais de armazenamento, assim como um índice de livro permite localizar capítulos rapidamente.

Você Sabia?

Uma tabela com **10 milhões de registros** sem índice pode levar **30 segundos** para uma consulta simples. Com um índice B-tree adequado, a mesma consulta retorna em **menos de 10 milissegundos** — uma melhoria de **3.000 vezes**.

Quando Indexar?

- **Sim:** Colunas usadas frequentemente em WHERE, JOIN e ORDER BY.
- **Sim:** Chaves estrangeiras que participam de junções.
- **Não:** Tabelas muito pequenas (o overhead do índice supera o ganho).
- **Cuidado:** Cada índice ocupa espaço e **torna INSERT/UPDATE mais lento** — equilíbrio é a chave.

O que é Índice?

Estrutura de Índice

Valores de Coluna

Os pontos de dados específicos usados para indexar e recuperar informações.



Estrutura de Dados

A estrutura fundamental que permite a organização e recuperação eficiente de dados.



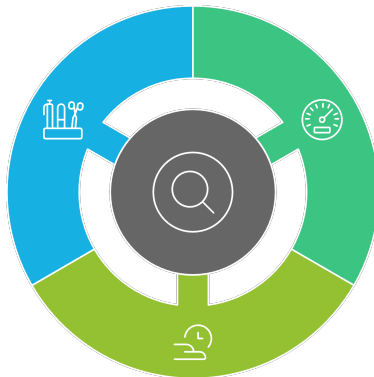
Acesso Rápido

A capacidade de
Banco de Dados I

Benefícios do Uso de Índices

Ordenação

Auxilia na ordenação dos resultados sem sobrecarregar o servidor.



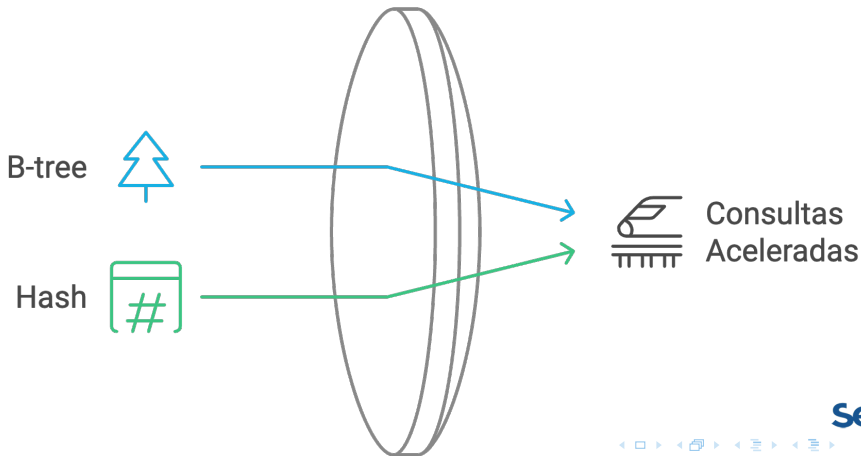
Desempenho

Aumenta a velocidade das operações de busca e filtragem.

Eficiência

Reduz o tempo de resposta das

Indexação em Bancos de Dados



- Criaremos uma tabela chamada Produtos com três colunas (id, nome, preco). Vamos inserir um número significativo de registros para simular uma tabela realista.

```
1 CREATE TABLE Produtos (  
2     id INT PRIMARY KEY AUTO_INCREMENT,  
3     nome VARCHAR(100),  
4     preco DECIMAL(10, 2)  
5 ) ENGINE=InnoDB;  
6  
7 INSERT INTO Produtos (nome, preco)  
8 SELECT CONCAT('Produto', FLOOR(RAND() * 1000)), RAND() * 1000  
9 FROM information_schema.tables  
10 LIMIT 1000;
```

```
1 -- Consulta sem índice na coluna 'preco'  
2 SELECT * FROM Produtos WHERE preco BETWEEN 100 AND 200;
```


- O índice B-tree (ou árvore B) é o tipo de índice padrão em muitos sistemas de banco de dados, incluindo o MySQL. Esse índice organiza os dados em uma estrutura de árvore balanceada, permitindo uma busca eficiente em grandes conjuntos de dados.

```
1 -- Adicionar um índice B-tree na coluna 'preco'
2 CREATE INDEX idx_preco ON Produtos(preco);
```

```
1 -- Consulta com índice na coluna 'preco'
2 SELECT * FROM Produtos WHERE preco BETWEEN 100 AND 200;
```

- Criaremos uma tabela chamada Clientes com três colunas (id, nome, email). Vamos inserir um número significativo de registros para simular uma tabela realista.

```
1 CREATE TABLE Clientes (  
2     id INT PRIMARY KEY AUTO_INCREMENT,  
3     nome VARCHAR(100),  
4     email VARCHAR(100)  
5 ) ENGINE=Memory;  
6  
7 INSERT INTO Clientes (nome, email)  
8 SELECT CONCAT('Cliente', FLOOR(RAND() * 359)), CONCAT('cliente', FLOOR(RAND() * 359),  
9     '@dominio.com')  
0 FROM information_schema.tables  
LIMIT 359;
```

```
1 -- Consulta sem índice hash  
2 SELECT * FROM Clientes WHERE email = 'cliente45@dominio.com';
```

- O índice Hash usa uma função de hashing para mapear valores da coluna para uma localização específica. Esse tipo de índice é muito rápido para buscas de igualdade (=), mas não é eficiente para buscas por intervalo (BETWEEN, >, <).

```
1 -- Adicionando índice hash na coluna 'email'
2 CREATE INDEX idx_email_hash ON Clientes(email) USING HASH;
```

```
1 -- Consulta com índice hash na coluna 'email'
2 SELECT * FROM Clientes WHERE email = 'cliente45@dominio.com';
```

- 1 Uma transação é um conjunto de operações SQL que são executadas como uma unidade única. Ou seja, todas as operações dentro da transação devem ser completadas com sucesso para que as alterações sejam efetivadas no banco de dados. Caso ocorra um erro durante a transação, o banco de dados pode reverter (rollback) todas as alterações, garantindo que nenhuma mudança parcial permaneça.
- 2 Uma transação normalmente envolve operações de inserção, atualização ou exclusão e é muito útil para garantir a integridade dos dados, especialmente em sistemas que exigem operações complexas e relacionadas.

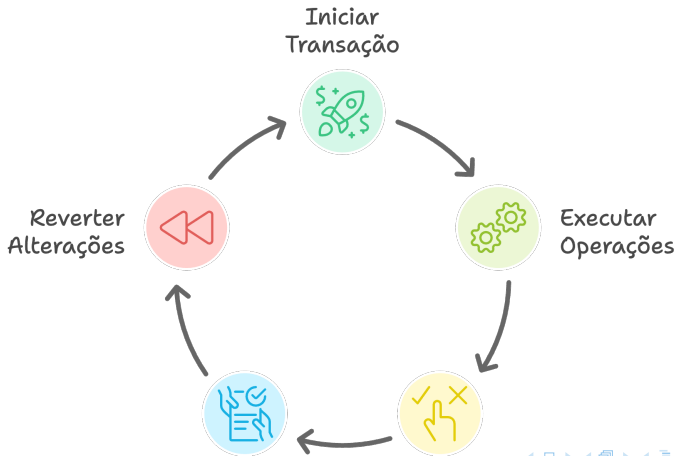
Caso Real: Pix e Transações

O **Pix**, sistema de pagamentos instantâneos do Banco Central, processa mais de **150 milhões de transações por dia**. Cada transferência é uma transação ACID que garante: o dinheiro **sai de uma conta e entra em outra** de forma atômica — ou ambas as operações acontecem, ou nenhuma.

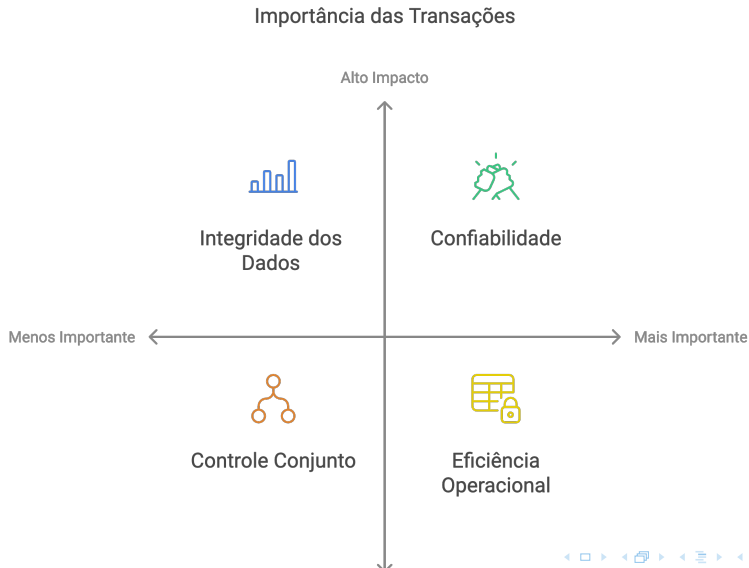
Por que ACID importa?

- **Sem Atomicidade:** O dinheiro poderia sair de sua conta e nunca chegar ao destino.
- **Sem Consistência:** Saldos negativos poderiam surgir sem verificação de regras.
- **Sem Isolamento:** Duas transferências simultâneas poderiam “consumir” o mesmo saldo.
- **Sem Durabilidade:** Uma queda de energia apagaria sua transferência confirmada.

Ciclo de Vida da Transação em Bancos de Dados



Por que as transações são importante?



- No MySQL, você pode iniciar uma transação usando comandos específicos e controlá-la com **COMMIT** e **ROLLBACK**:

```
1 START TRANSACTION; -- Inicia uma nova transação
2
3 -- Operações de transação, como INSERT, UPDATE, DELETE
4
5 COMMIT; -- Confirma as operações da transação
```

- Se algum problema ocorrer e você precisar cancelar as operações realizadas dentro da transação, pode usar **ROLLBACK**:

```
1 ROLLBACK; -- Reverte todas as operações realizadas na transação
```


Transação de Banco de Dados

Consistência

Mantém a integridade do banco de dados, garantindo que as transações levem a estados válidos.

Isolamento

Garante que as transações sejam executadas de forma independente, sem interferência.

Atomicidade

Garante que todas as operações em uma transação sejam concluídas com sucesso ou não sejam realizadas.

Durabilidade

Garante que, uma vez que uma transação é confirmada, ela permaneça permanente.



- As propriedades ACID (**A**tomicidade, **C**onsistência, **I**solamento e **D**urabilidade) são fundamentais para garantir que uma transação seja executada corretamente e que o banco de dados mantenha a integridade dos dados, mesmo em casos de falhas ou problemas.

Situação problema:

- Imagine que você está realizando uma transferência bancária entre duas contas. A transação precisa subtrair o valor de uma conta e adicionar à outra. Se uma das operações falhar, nenhuma das duas deve ser aplicada.

```
1 START TRANSACTION;  
2 UPDATE Conta SET saldo = saldo - 100 WHERE id = 1;  
3 UPDATE Conta SET saldo = saldo + 100 WHERE id = 2;  
4 COMMIT; -- Confirma as operações
```

- Se uma das operações falhar, você pode executar um ROLLBACK para desfazer a transação e evitar que apenas uma parte da operação seja aplicada.

Situação problema:

- Suponha que você tenha uma tabela de Pedidos com uma chave estrangeira para uma tabela Clientes. Se você tentar inserir um pedido com um cliente inexistente, o MySQL impedirá a operação para garantir a consistência.

```
1 START TRANSACTION;  
2 INSERT INTO Pedidos (id_cliente, valor) VALUES (999, 200); -- Cliente ID 999 não  
   existe  
3 COMMIT;
```

- A transação falhará devido à violação da chave estrangeira, e o MySQL impedirá a inconsistência

Níveis de Isolamento de Transação



Situação problema:

- Imagine duas transações simultâneas que tentam ler e modificar o mesmo dado. Dependendo do nível de isolamento, elas poderão ou não ver as alterações uma da outra antes de serem confirmadas (**COMMIT**).

```
1 SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
2 START TRANSACTION;  
3 -- Operações da transação  
4 COMMIT;
```

- A transação falhará devido à violação da chave estrangeira, e o MySQL impedirá a inconsistência

Durabilidade assegura que, uma vez que uma transação seja confirmada, suas alterações são permanentes, mesmo em caso de falha do sistema (como queda de energia ou reinicialização do servidor). O MySQL grava as alterações no log de transações, permitindo a recuperação dos dados em caso de falhas.

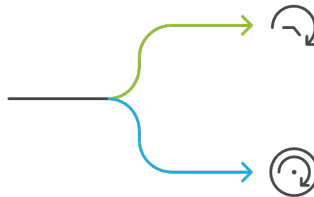
- Exemplo de Durabilidade

- Após o **COMMIT**, o MySQL grava as alterações em um log permanente. Mesmo se houver uma falha de energia imediatamente após o **COMMIT**, o MySQL poderá restaurar o banco de dados para o estado mais recente e consistente.

- Uma trigger é um conjunto de comandos SQL que é executado automaticamente quando uma determinada ação ocorre em uma tabela. As ações que podem disparar uma trigger incluem:
 - **INSERT**: Quando um novo registro é inserido na tabela.
 - **UPDATE**: Quando um registro existente é atualizado.
 - **DELETE**: Quando um registro é excluído.
- Uma trigger pode ser configurada para ser executada:
 - Antes da ação (**BEFORE**): Executa a trigger antes da execução do comando (ideal para validar dados antes da inserção ou atualização).
 - Depois da ação (**AFTER**): Executa a trigger após a execução do comando (bom para log de auditoria e cálculos).



Quando usar um trigger em um banco de dados?



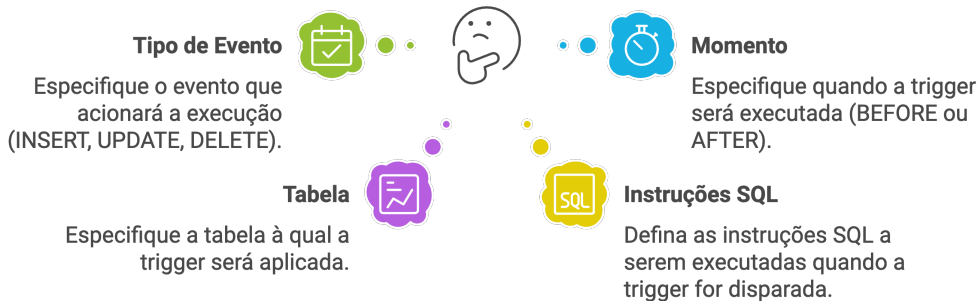
ANTES

Validar dados antes da inserção ou atualização.

DEPOIS

Registrar auditoria e realizar cálculos após a inserção ou atualização.

Como definir uma trigger no MySQL?



- Exemplo de sintaxe de criação de uma trigger:

```
1 CREATE TRIGGER nome_da_trigger
2 AFTER INSERT ON nome_da_tabela
3 FOR EACH ROW
4 BEGIN
5     -- Instruções SQL executadas pela trigger
6 END;
```

Aplicações Reais de Triggers

- **Auditoria:** Bancos como o **Itaú** usam triggers AFTER UPDATE para registrar automaticamente **toda alteração** em tabelas financeiras, criando trilhas de auditoria.
- **Validação:** E-commerces usam triggers BEFORE INSERT para verificar se o estoque é suficiente antes de confirmar um pedido.
- **Sincronização:** Triggers podem atualizar tabelas de cache ou enviar notificações quando dados críticos são modificados.

Cuidado com Triggers!

- Triggers executam de forma **invisível** — podem dificultar a depuração.
- Triggers encadeadas (trigger que dispara outra) podem causar **loops infinitos**.
- Use triggers para **lógica simples**; para regras complexas, prefira **stored procedures**.

O MySQL oferece ferramentas para controlar o acesso simultâneo aos dados, evitando conflitos e mantendo a integridade dos dados. Existem dois mecanismos principais de controle de concorrência:

Qual mecanismo de bloqueio usar?

Bloqueio Compartilhado

Permite que várias transações leiam o mesmo dado, mas impede modificações até que o bloqueio seja liberado.



Bloqueio Exclusivo

Apenas uma transação pode modificar o dado bloqueado, evitando que outras transações leiam ou modifiquem até a liberação.

Exemplo de bloqueio

- Vamos criar uma tabela Contas e ver como o bloqueio pode ser aplicado para gerenciar transações que alteram o saldo de uma conta.

```
1 CREATE TABLE Contas (  
2     id INT PRIMARY KEY AUTO_INCREMENT,  
3     nome VARCHAR(50),  
4     saldo DECIMAL(10, 2)  
5 );
```

- Inserindo dados para teste

```
1 INSERT INTO Contas (nome, saldo) VALUES ('Alice', 500.00), ('Bob', 300.00);
```

- Aplicando um bloqueio de leitura compartilhado

```
1 START TRANSACTION;  
2 SELECT saldo FROM Contas WHERE nome = 'Alice' LOCK IN SHARE MODE;
```

- Aplicando um bloqueio exclusivo

```
1 START TRANSACTION;  
2 SELECT saldo FROM Contas WHERE nome = 'Alice' FOR UPDATE;
```

- Para um bloqueio exclusivo, usamos **FOR UPDATE**, impedindo que outras transações leiam ou modifiquem o dado até a liberação do bloqueio.

Você Sabia?

Durante a **Black Friday 2023**, grandes varejistas brasileiros registraram picos de mais de **50.000 transações simultâneas por segundo**. Sem controle de concorrência adequado, problemas como **overselling** (vender mais do que o estoque) e **deadlocks** podem causar prejuízos milionários.

Estratégias do Mercado

- **Lock otimista** (versionamento): Usado por sistemas de e-commerce para alta performance — verifica conflitos apenas no momento do COMMIT.
- **Lock pessimista** (FOR UPDATE): Usado em sistemas bancários onde a **consistência** é mais importante que a velocidade.
- O **MySQL InnoDB** detecta **deadlocks automaticamente** e faz rollback da transação com menor custo.